OAVS • TGT COMPUTER SCIENCE

MCQ Question Bank

Dynamic Memory Allocation & Memory Functions

Covering: Dynamic Environment as Run-Time Environment • Dynamic Memory Management

Built-in Tools — **malloc()** & **free()** • Simple Programs Using malloc() & free()

170 Unique Exam-Oriented MCQs with Answers & Explanations

About This Question Bank

This question bank is strictly limited to the syllabus unit **"Dynamic Memory Allocation and Memory Functions"** as prescribed for the OAVS TGT Computer Science Teacher Examination: the dynamic environment as a run-time environment, the concept of dynamic memory management, the built-in dynamic memory tools of C — **malloc()** and **free()** — and simple programs demonstrating their use.

- Every question is derived only from this syllabus unit; no other C topics, functions (such as calloc/realloc), or advanced concepts have been introduced.
- A genuinely narrow, single-unit syllabus like this one has a natural ceiling on the number of truly distinct, non-repetitive, exam-quality questions that can be written — padding beyond that point would mean minor reworded duplicates rather than new questions. This bank contains **170 unique MCQs**, which is the practical maximum achieved here without resorting to artificial variation.
- Correct-answer options are balanced and randomised across A, B, C and D throughout the bank.
- Each question includes a short, exam-oriented explanation.

Table of Contents

Chapter 1	Dynamic Environment as Run-Time Environment	30 Qs	page 4
Chapter 2	Dynamic Memory Management	30 Qs	page 14
Chapter 3	The malloc() Function	50 Qs	page 23
Chapter 4	The free() Function	30 Qs	page 37
Chapter 5	Simple Programs Using malloc() and free()	30 Qs	page 47

Chapter 1 Dynamic Environment as Run-Time Environment 30 Questions

Q1 | **Definition** | *Dynamic Environment as Run-Time Environment*

What is a run-time environment?

A. The environment in which a program executes and its memory requirements are handled during execution

B. The environment used only to compile the source code

C. The environment used only to write the program code

D. The environment used only for debugging syntax errors

Ans: A

> **Explanation:** A run-time environment refers to the environment in which a program actually runs, where memory and resources are managed during execution.

Q2 | **Concept** | *Dynamic Environment as Run-Time Environment*

The term “dynamic” in a dynamic environment refers to:

A. Memory requirements fixed permanently before compilation

B. Variables that cannot be changed once declared

C. Memory requirements that can change or be decided while the program is running

D. Functions that execute only once during compilation

Ans: C

> **Explanation:** “Dynamic” means memory needs are determined and can change during program execution, not fixed in advance.

Q3 | **Concept** | *Dynamic Environment as Run-Time Environment*

A run-time environment comes into existence:

A. When the source code is written

B. When the program is compiled

C. When the program starts executing

D. When the program is saved to disk

Ans: C

> **Explanation:** The run-time environment exists only while the program is actually executing, not during writing or compiling.

Q4 | **Comparison** | *Dynamic Environment as Run-Time Environment*

How does compile-time memory allocation differ from run-time memory allocation?

A. Compile-time allocation is decided after execution ends

B. Compile-time allocation fixes memory size before execution, while run-time allocation decides memory size during execution

C. Run-time allocation is always fixed and cannot change

D. Both are decided at the same stage of the program

Ans: B

Explanation: Compile-time memory is fixed before the program runs, whereas run-time memory is decided dynamically as the program executes.

Q5 | **Scenario** | *Dynamic Environment as Run-Time Environment*

A program asks the user to enter the number of students and then needs to store their marks. Why is a dynamic environment useful here?

A. Because compile-time memory is always sufficient

B. Because the exact memory required is known only at run time based on user input

C. Because arrays cannot store marks

D. Because dynamic environments do not need memory

Ans: B

Explanation: Since the number of students is unknown until the user enters it, memory must be allocated at run time.

Q6 | **Concept** | *Dynamic Environment as Run-Time Environment*

What creates the need for dynamic memory allocation?

A. Memory requirements that never change

B. The need to reduce the size of the source code

C. The need to avoid using variables

D. Memory requirements that are unknown at compile time

Ans: D

Explanation: When memory requirements cannot be determined in advance, dynamic memory allocation is needed to allocate memory at run time.

Q7 | **Limitation of Static Memory** | *Dynamic Environment as Run-Time Environment*

What is a major drawback of using fixed-size (static) memory allocation when data size is unpredictable?

A. It always allocates exactly the right amount of memory

B. It automatically adjusts memory size at run time

C. It removes the need for a run-time environment

D. It may waste memory or fail to accommodate all the required data

Ans: D

> **Explanation:** Fixed-size allocation can either waste memory (if oversized) or be insufficient (if undersized) when the actual data size varies.

Q8 | **Concept** | *Dynamic Environment as Run-Time Environment*

In a dynamic environment, memory management decisions such as how much memory to allocate are made:

A. During program execution

B. Before the program is written

C. During compilation only

D. After the program is deleted

Ans: A

> **Explanation:** A dynamic environment makes memory-related decisions while the program is actually running.

Q9 | **Fill in the Blank** | *Dynamic Environment as Run-Time Environment*

The _____ is the memory area typically used for memory that is allocated during program execution.

A. stack

B. heap

C. register

D. code segment

Ans: B

> **Explanation:** Dynamically allocated memory in C is taken from the heap, unlike local variables which use the stack.

Q10 | **Statement Based** | *Dynamic Environment as Run-Time Environment*

Which statement about a run-time environment is correct?

A. It allows a program to allocate memory only before it starts running

B. It prevents a program from using any memory during execution

C. It is identical to the compile-time environment

D. It allows a program to allocate memory based on conditions known only while running

Ans: D

> **Explanation:** The run-time environment supports allocation decisions that depend on conditions discovered only during execution.

Q11 | **Conceptual** | *Dynamic Environment as Run-Time Environment*

Which of these is an example of a run-time memory requirement?

A. The number of array elements is decided based on user input during execution

B. The size of an int variable as defined by the C standard

C. The name of the source code file

D. The total number of lines in the compiled program

Ans: A

> **Explanation:** When array size depends on input received while the program runs, it is a run-time memory requirement.

Q12 | **Conceptual** | *Dynamic Environment as Run-Time Environment*

Compile-time memory allocation is best described as:

A. Memory whose size is decided only after the program finishes running

B. Memory that is allocated using malloc()

C. Memory that changes size while the program runs

D. Memory whose size is fixed and known before the program executes

Ans: D

> **Explanation:** Compile-time allocation fixes the memory size in advance, before execution begins.

Q13 | **Assertion-Reason** | *Dynamic Environment as Run-Time Environment*

Assertion (A): Dynamic memory allocation is performed at run time.
Reason (R): Functions like malloc() are executed while the program is running, not during compilation.

A. A is true but R is false

B. A is false but R is true

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: C

> **Explanation:** Dynamic allocation happens at run time because functions like malloc() execute during program execution, which correctly explains the assertion.

Q14 | **Basic** | *Dynamic Environment as Run-Time Environment*

Which environment is directly concerned with how a program behaves and uses memory while it is running?

A. Editing environment

B. Compilation environment

C. Run-time environment

D. Documentation environment

Ans: C

> **Explanation:** The run-time environment governs a program's behavior and memory usage during actual execution.

Q15 | **Conceptual** | *Dynamic Environment as Run-Time Environment*

Why can't all memory requirements always be decided at compile time?

A. Because some requirements, such as user input size, are known only when the program runs

B. Because compilers cannot allocate any memory

C. Because compile-time memory is always unlimited

D. Because C does not support compile-time memory allocation

Ans: A

> **Explanation:** Certain data (like user input) is unknown until run time, so memory for it cannot be fixed at compile time.

Q16 | **Statement Based** | *Dynamic Environment as Run-Time Environment*

Which of the following best supports the need for a dynamic environment in programming?

A. Programs often need to handle data whose exact size is not known in advance

B. Programs never need more memory than declared at compile time

C. All programs use a fixed amount of memory regardless of input

D. Compile-time allocation is always more efficient than run-time allocation

Ans: A

> **Explanation:** A dynamic environment is needed because many programs must handle data sizes that are unknown until run time.

Q17 | **Match the Following** | *Dynamic Environment as Run-Time Environment*

Match the type of memory allocation with its correct timing:
1. Static allocation 2. Dynamic allocation
A. Decided before program runs B. Decided while the program runs

A. 1-B, 2-A

B. 1-A, 2-B

C. Both 1 and 2 match with A

D. Both 1 and 2 match with B

Ans: B

> **Explanation:** Static allocation size is fixed before execution (compile time), while dynamic allocation size is decided during execution (run time).

Q18 | **Conceptual** | *Dynamic Environment as Run-Time Environment*

A key characteristic of a run-time environment is that it:

A. Never changes once the program starts

B. Only manages the compilation process

C. Cannot use functions like malloc() or free()

D. Can adapt memory usage based on conditions occurring during execution

Ans: D

> **Explanation:** The run-time environment is adaptive, allowing memory usage to change according to conditions that arise during execution.

Q19 | **Tricky** | *Dynamic Environment as Run-Time Environment*

Which of the following is decided only at run time and NOT at compile time?

A. The exact amount of memory to allocate for user-entered data

B. The size of the int data type

C. The syntax rules of the C language

D. The number of keywords in C

Ans: A

> **Explanation:** The int size and language syntax are fixed by the compiler/standard, but memory needed for user-entered data is determined at run time.

Q20 | **Application Based** | *Dynamic Environment as Run-Time Environment*

A program must store an unknown number of employee records provided during execution. Which type of memory allocation environment is required?

A. Static (compile-time) memory allocation only

B. Dynamic (run-time) memory allocation

C. No memory allocation is required

D. Allocation decided before the program is written

Ans: B

> **Explanation:** Since the number of records is unknown until execution, a dynamic environment using run-time allocation is required.

Q21 | **Concept** | *Dynamic Environment as Run-Time Environment*

Which statement correctly distinguishes dynamic environment from static environment?

A. Both environments make decisions only after the program ends

B. A static environment allocates memory only using malloc()

C. In a dynamic environment, memory decisions are made during execution; in a static environment, they are fixed beforehand

D. A dynamic environment never uses memory at all

Ans: C

> **Explanation:** The core distinction is timing: dynamic environments decide memory usage during execution, static environments fix it earlier.

Q22 | Basic | *Dynamic Environment as Run-Time Environment*

The need for a dynamic environment mainly arises due to:

A. The need to reduce compiler errors

B. Variability in data size that is only known during program execution

C. The need to increase source code readability

D. The requirement to avoid using variables in a program

Ans: B

> **Explanation:** Data whose size varies and is known only at run time creates the need for a dynamic environment.

Q23 | Statement Based | *Dynamic Environment as Run-Time Environment*

Which of the following is true regarding compile-time memory requirements?

A. They are determined before the program starts executing

B. They are determined after the program finishes executing

C. They always exceed run-time memory requirements

D. They are irrelevant to program execution

Ans: A

> **Explanation:** Compile-time requirements are fixed and known before execution starts.

Q24 | Conceptual | *Dynamic Environment as Run-Time Environment*

In the context of a dynamic environment, “run time” refers to:

A. The period during which the source code is being typed

B. The period during which the program is actually executing

C. The period during which the compiler checks for syntax errors

D. The period before the program is written

Ans: B

> **Explanation:** Run time is the period when the program is actively executing instructions.

Q25 | **Fill in the Blank** | *Dynamic Environment as Run-Time Environment*

Programs that need to allocate memory based on values entered by the user require a _____ environment.

A. static

B. dynamic (run-time)

C. compile-time only

D. syntax-checking

Ans: B

> **Explanation:** Since the memory requirement depends on user input, it can only be resolved in a dynamic (run-time) environment.

Q26 | **Conceptual** | *Dynamic Environment as Run-Time Environment*

Which of these best explains why static arrays are sometimes inefficient?

A. Their fixed size may either waste memory or be insufficient for actual data needs

B. They always allocate the exact amount of memory needed

C. They automatically resize during execution

D. They are stored outside the memory of the program

Ans: A

> **Explanation:** Static arrays have a size fixed at compile time, which can lead to wasted or insufficient memory depending on actual data.

Q27 | **Difficult** | *Dynamic Environment as Run-Time Environment*

Which of the following scenarios does NOT require a dynamic environment?

A. Declaring a fixed-size array of 10 integers whose size never changes

B. Allocating memory for a number of records entered by the user

C. Allocating memory whose size depends on a value read during execution

D. Allocating memory for data whose quantity is unknown until run time

Ans: A

> **Explanation:** A fixed-size array with a size that never changes can be handled with compile-time (static) allocation; the others require a dynamic environment.

Q28 | **Statement Based** | *Dynamic Environment as Run-Time Environment*

Which statement is correct about the relationship between run-time environment and memory management?

A. The run-time environment has no role in memory management

B. The run-time environment provides the basis for managing memory dynamically during execution

C. Memory management occurs only at compile time in C

D. The run-time environment prevents memory allocation altogether

Ans: B

> **Explanation:** The run-time environment is what makes dynamic memory management possible, since allocation decisions occur while the program runs.

Q29 | **Basic** | *Dynamic Environment as Run-Time Environment*

Why is understanding the run-time environment important before learning dynamic memory management in C?

A. Because malloc() and free() operate only during compilation

B. Because run-time concepts are unrelated to memory functions

C. Because the run-time environment only affects variable naming

D. Because dynamic memory functions like malloc() and free() operate during run time

Ans: D

> **Explanation:** Since malloc() and free() work while the program executes, understanding the run-time environment is essential to understanding them.

Q30 | **Conceptual** | *Dynamic Environment as Run-Time Environment*

Which best summarizes the concept of a “dynamic environment as a run-time environment”?

A. An environment where memory allocation and related decisions are handled while the program executes

B. An environment where all memory is allocated before the program starts

C. An environment used only for writing pseudocode

D. An environment that eliminates the need for memory altogether

Ans: A

> **Explanation:** The dynamic environment is essentially the run-time environment in which memory allocation decisions happen during execution.

Chapter 2 Dynamic Memory Management

30 Questions

Q31 | **Definition** | *Dynamic Memory Management*

What is dynamic memory management?

A. The process of allocating and releasing memory during program execution as needed

B. The process of allocating memory only before compilation

C. The process of permanently reserving memory that can never be released

D. The process of managing only static/global variables

Ans: A

> **Explanation:** Dynamic memory management involves allocating memory when needed and releasing it during program execution.

Q32 | **Concept** | *Dynamic Memory Management*

What is the main purpose of dynamic memory management?

A. To permanently allocate maximum memory at program start

B. To prevent a program from using any memory

C. To efficiently allocate memory only when it is required and release it when no longer needed

D. To allocate memory only for global variables

Ans: C

> **Explanation:** Its purpose is efficient use of memory by allocating on demand and freeing memory once it is no longer needed.

Q33 | **Concept** | *Dynamic Memory Management*

When is memory allocated in dynamic memory management?

A. Before the program is compiled

B. Only when the program is written

C. During program execution, when the need arises

D. After the program has finished executing

Ans: C

> **Explanation:** Dynamic memory is allocated at run time, exactly when the program needs it.

Q34 | **Concept** | *Dynamic Memory Management*

When should dynamically allocated memory be deallocated?

A. Immediately after it is allocated, regardless of use

B. When it is no longer needed by the program

C. Only when the computer is restarted

D. Only if the program never uses it

Ans: B

> **Explanation:** Memory should be released once its purpose is served, to avoid wasting resources.

Q35 | **Advantage** | *Dynamic Memory Management*

Which of the following is an advantage of dynamic memory management?

A. It removes the need for any memory during execution

B. Memory is used more efficiently, since it is allocated only as required

C. It guarantees the program cannot have errors

D. It eliminates the use of pointers entirely

Ans: B

> **Explanation:** One key advantage is efficient use of memory since allocation matches actual need.

Q36 | **Advantage** | *Dynamic Memory Management*

Dynamic memory management is particularly useful when:

A. The amount of memory needed is always fixed and known in advance

B. The program does not use any variables

C. The amount of memory needed is not known until the program runs

D. The program is very small and simple

Ans: C

> **Explanation:** It is most beneficial when memory requirements vary and are determined during execution.

Q37 | **Limitation** | *Dynamic Memory Management*

What is a common limitation of dynamic memory management in C?

A. Memory is automatically freed by the compiler

B. It cannot be used with pointers

C. It works only with global variables

D. The programmer must manually allocate and free memory, and forgetting to do so causes errors

Ans: D

> **Explanation:** Unlike some languages, C requires the programmer to manually manage memory, which can lead to memory leaks or errors if done incorrectly.

Q38 | **Limitation** | *Dynamic Memory Management*

What problem can occur if dynamically allocated memory is never freed?

A. The program automatically frees it after 10 seconds

B. The compiler reports a syntax error

C. Memory leak, where unused memory remains reserved and unavailable for reuse

D. The variable is deleted from the source code

Ans: C

> **Explanation:** Not freeing memory that is no longer needed causes a memory leak, wasting available memory.

Q39 | **Basic** | *Dynamic Memory Management*

In C, from which region of memory is dynamically allocated memory typically obtained?

A. Heap

B. Stack

C. CPU registers

D. Code (text) segment

Ans: A

> **Explanation:** Dynamically allocated memory in C comes from the heap, managed using functions like malloc().

Q40 | **Basic** | *Dynamic Memory Management*

Which header file must be included in a C program to use dynamic memory management functions?

A. stdio.h

B. string.h

C. math.h

D. stdlib.h

Ans: D

> **Explanation:** Functions such as malloc() and free() are declared in the stdlib.h header file.

Q41 | **Statement Based** | *Dynamic Memory Management*

Which of the following statements about dynamic memory management is correct?

A. It removes all responsibility for memory handling from the programmer

B. It only works with arrays of fixed size

C. It cannot be used in C programs

D. It gives the programmer control over allocating and releasing memory during execution

Ans: D

> **Explanation:** Dynamic memory management allows the programmer to explicitly control when memory is allocated and released.

Q42 | **Scenario** | *Dynamic Memory Management*

A program needs to store data whose size depends on a value entered by the user at run time. What is the most suitable approach?

A. Use dynamic memory management to allocate memory based on the user's input

B. Always declare the largest possible fixed-size array

C. Avoid storing the data altogether

D. Use only compile-time constants

Ans: A

> **Explanation:** Dynamic memory management allows memory to be allocated exactly according to the run-time requirement.

Q43 | **Fill in the Blank** | *Dynamic Memory Management*

Memory that is allocated and released during the execution of a program is known as _____ memory.

A. static

B. global

C. constant

D. dynamic

Ans: D

> **Explanation:** Memory handled during execution through allocation and deallocation is called dynamic memory.

Q44 | **Assertion-Reason** | *Dynamic Memory Management*

Assertion (A): Dynamic memory management improves memory utilization.
Reason (R): Memory is allocated only when required and released after use.

A. Both A and R are true, and R correctly explains A

B. A is true but R is false

C. A is false but R is true

D. Both A and R are false

Ans: A

> **Explanation:** Allocating memory only when needed and releasing it after use directly leads to better memory utilization, so R correctly explains A.

Q45 | Comparison | *Dynamic Memory Management*

How does dynamic memory differ from static memory in terms of allocation timing?

A. Dynamic memory is fixed at compile time, while static memory changes during execution

B. Both are allocated at exactly the same time

C. Neither is related to program execution

D. Dynamic memory is allocated and released during execution, while static memory is fixed at compile time

Ans: D

Explanation: Dynamic memory allocation and deallocation happen at run time, unlike static memory which is fixed before execution.

Q46 | Basic | *Dynamic Memory Management*

What must a programmer typically do after finishing use of dynamically allocated memory in C?

A. Do nothing, as C automatically frees all memory

B. Restart the program

C. Recompile the program

D. Explicitly free the memory using free()

Ans: D

Explanation: In C, the programmer must explicitly release dynamically allocated memory using free() when it is no longer needed.

Q47 | Statement Based | *Dynamic Memory Management*

Which of the following is NOT true about dynamic memory management?

A. It allows memory to be allocated based on run-time needs

B. It automatically deallocates memory without any action from the programmer in C

C. It requires the use of functions such as malloc() and free()

D. It can lead to memory leaks if not handled carefully

Ans: B

Explanation: In C, memory is not automatically deallocated; the programmer must call free() explicitly, making this statement false.

Q48 | **Conceptual** | *Dynamic Memory Management*

Which of the following best describes the core operations involved in dynamic memory management?

A. Compiling the program and linking libraries

B. Allocating memory when needed and deallocating it when no longer required

C. Declaring global variables and constants

D. Writing comments and documentation

Ans: B

> **Explanation:** The two core operations are allocation (getting memory) and deallocation (releasing memory).

Q49 | **Difficult** | *Dynamic Memory Management*

Which situation illustrates a limitation of dynamic memory management rather than an advantage?

A. Memory is allocated exactly according to run-time needs

B. Memory usage adapts to varying data sizes

C. A programmer forgets to release memory, resulting in wasted memory during program execution

D. Unused memory can be released back for reuse

Ans: C

> **Explanation:** Forgetting to release memory shows the risk/limitation of manual dynamic memory management, unlike its advantages of flexibility and efficiency.

Q50 | **Scenario** | *Dynamic Memory Management*

Why might a program using only static memory fail when handling large, unpredictable amounts of user data?

A. Because static memory can always resize itself automatically

B. Because static memory has a fixed size decided in advance and cannot adjust to unexpected data sizes

C. Because static memory is stored on the heap

D. Because static memory does not exist in C

Ans: B

> **Explanation:** Static memory size cannot change during execution, so it may be insufficient for unpredictable, large data.

Q51 | **Basic** | *Dynamic Memory Management*

Dynamic memory management primarily helps programs to:

A. Avoid using any pointers in the program

B. Use memory efficiently according to actual run-time requirements

C. Increase the size of the source code

D. Prevent the program from compiling

Ans: B

Explanation: It enables efficient memory use tailored to what is actually needed while the program executes.

Q52 | **Statement Based** | *Dynamic Memory Management*

Which of the following correctly relates dynamic memory management to the run-time environment?

A. Dynamic memory management has no connection to the run-time environment

B. Dynamic memory management occurs before the run-time environment exists

C. The run-time environment prevents dynamic memory management

D. Dynamic memory management operates within the run-time environment, since allocation and deallocation happen during execution

Ans: D

Explanation: Since allocation and deallocation happen while the program runs, dynamic memory management is part of the run-time environment.

Q53 | **Tricky** | *Dynamic Memory Management*

Which of the following is a correct statement about dynamically allocated memory that is not freed before a program terminates normally on most modern operating systems?

A. The memory remains permanently lost even after the process ends

B. The memory is automatically compacted into the stack

C. The operating system reclaims the memory when the process ends, but relying on this is considered poor programming practice

D. The C compiler prevents the program from terminating

Ans: C

Explanation: While most operating systems reclaim memory after a process ends, it is still good practice to explicitly free memory to avoid leaks during execution.

Q54 | **Basic** | *Dynamic Memory Management*

Which of these is a typical advantage of dynamic over static memory allocation?

A. It never requires any function calls

B. It uses less code than static allocation in every case

C. It allows memory size to be decided based on actual run-time requirements

D. It eliminates the need for a run-time environment

Ans: C

> **Explanation:** Dynamic allocation's key advantage is flexibility — the size can match the real, run-time need.

Q55 | **Conceptual** | *Dynamic Memory Management*

In dynamic memory management, “deallocation” refers to:

A. Releasing previously allocated memory so it can be reused

B. Allocating new memory for a variable

C. Declaring a variable at compile time

D. Compiling the program without errors

Ans: A

> **Explanation:** Deallocation means giving back memory that was previously allocated so it becomes available again.

Q56 | **Statement Based** | *Dynamic Memory Management*

Which statement best reflects a limitation of manual dynamic memory management in C?

A. The programming language automatically manages all memory perfectly

B. There is no risk of errors when using dynamic memory

C. Dynamic memory management cannot be used with functions

D. The programmer is responsible for correctly allocating and freeing memory, and mistakes can cause errors

Ans: D

> **Explanation:** In C, manual memory management places responsibility on the programmer, and mistakes can lead to problems like leaks or invalid access.

Q57 | **Application Based** | *Dynamic Memory Management*

Which scenario best demonstrates proper dynamic memory management?

A. Allocating memory once and never using or freeing it

B. Freeing memory before allocating it

C. Allocating memory for data needed, using it, and then freeing it once it is no longer required

D. Allocating memory repeatedly without ever using it

Ans: C

> **Explanation:** Proper dynamic memory management involves allocation, appropriate use, and timely deallocation.

Q58 | **Fill in the Blank** | *Dynamic Memory Management*

Failing to release memory that is no longer needed can result in a _____.

A. compilation error

B. syntax warning

C. stack overflow only

D. memory leak

Ans: D

> **Explanation:** Unused memory that is never freed accumulates and causes a memory leak.

Q59 | **Basic** | *Dynamic Memory Management*

Which pair of operations forms the foundation of dynamic memory management?

A. Compilation and linking

B. Allocation and deallocation

C. Declaration and initialization only

D. Input and output

Ans: B

> **Explanation:** Dynamic memory management is fundamentally about allocating memory when needed and deallocating it when done.

Q60 | **Conceptual** | *Dynamic Memory Management*

Why is dynamic memory management considered essential in programs handling variable-sized data?

A. Because it forces the program to always use the maximum possible memory

B. Because it lets the program request only as much memory as is actually needed at run time

C. Because it removes the need for any memory at all

D. Because it only works with fixed-size data

Ans: B

> **Explanation:** Its core benefit is matching memory allocation to the real, variable needs discovered during execution.

Chapter 3 The malloc() Function

50 Questions

Q61 | **Purpose** | *The malloc() Function*

What is the purpose of the malloc() function in C?

A. To release previously allocated memory

B. To print formatted output to the console

C. To dynamically allocate a specified number of bytes of memory during program execution

D. To compare two strings

Ans: C

> **Explanation:** malloc() is used to dynamically allocate a block of memory of a given size during program execution.

Q62 | **Syntax** | *The malloc() Function*

Which header file must be included to use malloc()?

A. stdio.h

B. conio.h

C. string.h

D. stdlib.h

Ans: D

> **Explanation:** malloc() is declared in the stdlib.h header file.

Q63 | **Syntax** | *The malloc() Function*

What is the general syntax for calling malloc()?

A. ptr = (cast_type*) malloc(size_in_bytes);

B. ptr = malloc[size_in_bytes];

C. malloc(ptr, size_in_bytes);

D. ptr = malloc.size(size_in_bytes);

Ans: A

> **Explanation:** malloc() is called as malloc(size), with the returned pointer typically cast to the required pointer type.

Q64 | **Argument** | *The malloc() Function*

What does the argument passed to malloc() represent?

A. The number of variables to declare

B. The number of bytes of memory to allocate

C. The name of the variable being allocated

D. The data type of the pointer

Ans: B

> **Explanation:** malloc() takes a single argument specifying the number of bytes to allocate.

Q65 | Return Value | *The malloc() Function*

What does malloc() return on successful allocation?

A. The exact value stored in the allocated memory

B. A pointer to the first byte of the allocated memory block

C. An integer representing the memory address as a number

D. The size of memory allocated

Ans: B

> **Explanation:** On success, malloc() returns a pointer (of type void*) pointing to the beginning of the allocated block.

Q66 | Return Value | *The malloc() Function*

What is the data type of the pointer returned by malloc()?

A. int* only

B. char* only

C. void*

D. float* only

Ans: C

> **Explanation:** malloc() returns a generic pointer of type void*, which can be assigned to any pointer type.

Q67 | Failure Case | *The malloc() Function*

What does malloc() return if it fails to allocate the requested memory?

A. NULL

B. 0.0

C. -1

D. An uninitialized pointer with garbage value

Ans: A

> **Explanation:** If malloc() cannot allocate the requested memory, it returns NULL.

Q68 | Concept | *The malloc() Function*

Why might malloc() fail to allocate memory?

A. If the program has too few variables

B. If the source code has comments

C. If there is not enough free memory available in the heap

D. If the compiler is outdated

Ans: C

> **Explanation:** malloc() fails when the system does not have enough available memory to satisfy the request.

Q69 | **Best Practice** | *The malloc() Function*

After calling malloc(), what should a programmer always check?

A. Whether the pointer's address is even or odd

B. Whether the pointer was declared as global

C. Whether the pointer's name starts with a letter

D. Whether the returned pointer is NULL, to detect allocation failure

Ans: D

> **Explanation:** Checking for NULL ensures the program can detect and handle failed memory allocation instead of crashing.

Q70 | **Size Calculation** | *The malloc() Function*

When allocating memory for an array of 10 integers using malloc(), which expression correctly calculates the required size?

A. sizeof(10)

B. 10 * sizeof(int)

C. 10 + sizeof(int)

D. sizeof(int) / 10

Ans: B

> **Explanation:** The number of elements multiplied by the size of one element gives the total bytes needed.

Q71 | **Code Snippet** | *The malloc() Function*

What does the following statement do?

```
int *p = (int*) malloc(5 * sizeof(int));
```

A. Allocates memory for 5 characters only

B. Declares 5 separate integer variables named p

C. Frees memory pointed to by p

D. Allocates memory for 5 integers and stores the starting address in pointer p

Ans: D

> **Explanation:** This allocates space for 5 integers (5 * sizeof(int) bytes) and assigns the address to p.

Q72 | Casting | *The malloc() Function*

Why is the return value of malloc() often cast to a specific pointer type, such as (int*)?

A. To make the pointer type match the type of data it will point to, improving clarity

B. Because malloc() cannot compile without a cast in any C compiler

C. Because casting changes the amount of memory allocated

D. Because malloc() returns an int by default

Ans: A

Explanation: Casting the void* return value clarifies the intended pointer type, though it is not strictly required in standard C.

Q73 | Casting / Tricky | *The malloc() Function*

In standard C (not C++), is casting the return value of malloc() mandatory?

A. Yes, the program will not compile without an explicit cast in C

B. Yes, malloc() only works with cast values

C. No, because void* is automatically converted to any pointer type in C

D. No, because malloc() does not return a pointer

Ans: C

Explanation: In C, void* is implicitly convertible to other pointer types, so casting malloc()'s return value is optional, though commonly done for clarity.

Q74 | Memory Content | *The malloc() Function*

What is the initial content of memory allocated by malloc()?

A. Automatically set to zero

B. Automatically set to the value 1

C. Filled with the string "NULL"

D. Uninitialized (contains garbage/indeterminate values)

Ans: D

Explanation: malloc() does not initialize the allocated memory; it may contain leftover, indeterminate values.

Q75 | **Common Mistake** | *The malloc() Function*

What is a common mistake when using malloc()?

A. Not checking whether the returned pointer is NULL before using it

B. Including stdlib.h before using malloc()

C. Passing the correct size in bytes to malloc()

D. Storing the returned pointer in a variable

Ans: A

> **Explanation:** Failing to check for NULL can cause the program to crash if allocation fails and the pointer is dereferenced.

Q76 | **Common Mistake** | *The malloc() Function*

What problem occurs if a program uses a pointer returned by a failed malloc() call without checking for NULL?

A. The program automatically retries the allocation

B. The compiler generates an error before running

C. The pointer is automatically converted to a valid address

D. The program may crash or behave unpredictably due to dereferencing a NULL pointer

Ans: D

> **Explanation:** Dereferencing a NULL pointer leads to undefined behavior, often causing the program to crash.

Q77 | **Proper Use** | *The malloc() Function*

Which of the following is the correct way to allocate memory for a single integer using malloc()?

A. int *p = malloc(int);

B. int p = malloc(sizeof(int));

C. int *p = malloc();

D. int *p = (int*) malloc(sizeof(int));

Ans: D

> **Explanation:** The correct usage passes sizeof(int) as the size argument and assigns the result to an integer pointer.

Q78 | **Output / Prediction** | *The malloc() Function*

What happens when the statement malloc(0) is executed?

A. It always causes a compilation error

B. It always allocates exactly one byte of memory in every compiler

C. The behavior is implementation-defined; it may return NULL or a valid pointer that should not be dereferenced

D. It always crashes the program immediately

Ans: C

Explanation: Calling malloc(0) has implementation-defined behavior according to the C standard.

Q79 | **Concept** | *The malloc() Function*

Which statement about malloc() and initialization is correct?

A. malloc() always initializes memory to zero

B. malloc() initializes memory to the value of the pointer variable

C. malloc() allocates memory but does not initialize its contents

D. malloc() initializes memory only for integer pointers

Ans: C

Explanation: Unlike some other allocation functions, malloc() leaves the allocated memory uninitialized.

Q80 | **Function Based** | *The malloc() Function*

Which function call correctly allocates memory for an array of 20 float elements?

A. (float*) malloc(20)

B. (float*) malloc(sizeof(float))

C. (float*) malloc(float * 20)

D. (float*) malloc(20 * sizeof(float))

Ans: D

Explanation: The total bytes required is the number of elements multiplied by the size of each float.

Q81 | **Error Identification** | *The malloc() Function*

Identify the error in the following statement:
int *p = malloc(sizeof(int))

A. malloc() cannot allocate memory for int pointers

B. sizeof(int) is invalid syntax

C. The pointer p is declared incorrectly

D. The statement is missing a semicolon at the end

Ans: D

Explanation: Every C statement must end with a semicolon; this statement is otherwise syntactically valid.

Q82 | **Scenario** | *The malloc() Function*

A program needs memory for storing 100 characters at run time. Which malloc() call is most appropriate?

A. (char*) malloc(100 * sizeof(char))

B. (char*) malloc(100.0)

C. (char*) malloc()

D. (int*) malloc(char)

Ans: A

Explanation: This correctly requests 100 bytes (since sizeof(char) is 1) for storing 100 characters.

Q83 | **Output / Prediction** | *The malloc() Function*

If malloc() successfully allocates memory, what value will the pointer variable hold?

A. The value zero

B. The number of bytes allocated

C. The starting address of the allocated memory block

D. The value NULL

Ans: C

Explanation: On success, the pointer holds the address where the allocated block begins.

Q84 | **Concept** | *The malloc() Function*

What is stored in the memory block allocated by malloc() before any values are assigned to it?

A. Garbage (indeterminate) values left over from previous use of that memory

B. Always the value 0

C. Always the value -1

D. The address of the pointer itself

Ans: A

Explanation: malloc() does not clear memory, so it may contain leftover data from earlier use.

Q85 | **Application** | *The malloc() Function*

Why is malloc() useful in a program that reads an unknown number of values from the user?

A. It automatically determines how many values the user will enter without any allocation

B. It prevents the user from entering more than 10 values

C. It removes the need to use pointers

D. It allows memory to be allocated for exactly the number of values needed, determined at run time

Ans: D

Explanation: malloc() lets the program request memory sized according to run-time input, which is essential for unknown-size data.

Q86 | **Tricky** | *The malloc() Function*

What happens if you call malloc() with a very large size that exceeds available memory?

A. malloc() automatically allocates as much memory as is available

B. malloc() returns NULL, indicating that the allocation failed

C. The compiler stops the program from running

D. The operating system crashes

Ans: B

> **Explanation:** When the requested size cannot be satisfied, malloc() fails and returns NULL rather than allocating a lesser amount.

Q87 | **Statement Based** | *The malloc() Function*

Which of these correctly describes malloc()'s role in C?

A. It reserves a contiguous block of memory of a specified size on the heap

B. It reserves memory on the stack for local variables

C. It compiles a C program into an executable

D. It declares a new function in the program

Ans: A

> **Explanation:** malloc() reserves a contiguous block of memory of the requested size from the heap.

Q88 | **Comparison** | *The malloc() Function*

Which of the following differentiates malloc() from a normal variable declaration like int x;?

A. malloc() always uses less memory than a normal declaration

B. malloc() allocates memory during execution, while a normal declaration allocates memory determined at compile time

C. A normal declaration also requires a size argument like malloc()

D. malloc() can only be used for character variables

Ans: B

> **Explanation:** malloc() defers memory allocation to run time, whereas a plain variable declaration is handled at compile time.

Q89 | **Fill in the Blank** | *The malloc() Function*

The malloc() function takes _____ as its only argument.

A. the name of the variable to store data in

B. the type of data to be stored

C. the initial value to be stored

D. the number of bytes to allocate

Ans: D

> **Explanation:** malloc() requires only the number of bytes to allocate; it has no knowledge of variable names or types.

Q90 | **Assertion-Reason** | *The malloc() Function*

Assertion (A): It is good practice to check the pointer returned by malloc() before using it. Reason (R): malloc() may return NULL if memory allocation fails.

A. A is true but R is false

B. A is false but R is true

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: C

> **Explanation:** Checking for NULL is good practice precisely because malloc() can fail and return NULL, so R correctly explains A.

Q91 | **Program Tracing** | *The malloc() Function*

```
What is the output of the following code if malloc() succeeds?
int *p = (int*) malloc(sizeof(int));
*p = 25;
printf("%d", *p);
```

A. 25

B. Garbage value

C. 0

D. Compilation error

Ans: A

> **Explanation:** After allocation, the value 25 is stored at the location pointed to by p, so printing *p outputs 25.

Q92 | **Error Identification** | *The malloc() Function*

What is wrong with the following code?

```
int *p;
*p = 10;
```

A. The variable p should have been declared as float

B. Memory for p was never allocated using malloc() before dereferencing it

C. The value 10 cannot be assigned to an integer pointer

D. printf() is missing from the code

Ans: B

> **Explanation:** Dereferencing an uninitialized pointer that was never allocated memory leads to undefined behavior.

Q93 | **Concept** | *The malloc() Function*

Which of the following best explains why malloc() is called a “dynamic” memory allocation function?

A. Because it can only be called once in a program

B. Because it changes the value of existing variables automatically

C. Because it works only with dynamic-type variables in C

D. Because it allocates memory while the program is running, based on the size specified at that time

Ans: D

> **Explanation:** malloc() allocates memory at run time according to the size given, which is what makes it “dynamic”.

Q94 | **Basic** | *The malloc() Function*

How many arguments does the malloc() function accept?

A. Two (size and type)

B. One (the size in bytes)

C. Three (size, type, and initial value)

D. Zero

Ans: B

> **Explanation:** malloc() accepts a single argument representing the number of bytes to allocate.

Q95 | **Difficult** | *The malloc() Function*

Consider: char *str = (char*) malloc(10);. What does this statement allocate?

A. 10 characters of memory that automatically includes an extra null terminator beyond the 10 bytes

B. 10 bytes of memory, enough to store a string of up to 9 characters plus a null terminator

C. 10 integers of memory

D. Memory for exactly 10 pointers

Ans: B

> **Explanation:** malloc(10) allocates 10 bytes; since sizeof(char) is 1, this can hold a 9-character string plus a terminating null character within those 10 bytes.

Q96 | **Scenario** | *The malloc() Function*

A programmer wants to allocate memory for an array of 50 double values. Which statement is correct?

A. double *p = (double*) malloc(50 * sizeof(double));

B. double *p = (double*) malloc(50);

C. double p = (double*) malloc(50 * sizeof(double));

D. double *p = malloc(sizeof(double) - 50);

Ans: A

> **Explanation:** The correct call multiplies the number of elements by sizeof(double) to compute the total bytes needed.

Q97 | Tricky | *The malloc() Function*

If sizeof(int) is 4 bytes, how many bytes does malloc(4 * sizeof(int)) allocate?

A. 4 bytes

B. 8 bytes

C. 16 bytes

D. 20 bytes

Ans: C

Explanation: 4 * sizeof(int) = 4 * 4 = 16 bytes, since sizeof(int) is 4 on the assumed system.

Q98 | Statement Based | *The malloc() Function*

Which statement correctly compares malloc() allocation with array declaration such as int arr[10]?

A. Both always use exactly the same amount of memory in every case

B. malloc() allocation size can depend on a run-time value, whereas an ordinary array's size is generally fixed at compile time

C. Array declarations always require malloc() internally in C

D. malloc() cannot be used to allocate memory equivalent to an array

Ans: B

Explanation: A key distinction is that malloc()'s size argument can be a run-time expression, unlike a plain array's fixed compile-time size.

Q99 | Application | *The malloc() Function*

Which of the following is a valid reason to prefer malloc() over a fixed-size array in a program?

A. malloc() always executes faster than array declarations in every case

B. The exact number of elements needed is not known until the program runs

C. malloc() removes the need to use pointers

D. Fixed-size arrays cannot store integers

Ans: B

Explanation: malloc() is preferred when the required size is only known at run time, unlike a fixed-size array.

Q100 | **Program Tracing** | *The malloc() Function*

```
What will the following code print if malloc() fails and returns NULL, and no NULL
check is performed?
int *p = (int*) malloc(1000000000000);
*p = 5;
```

A. It will print 5 successfully

B. It will print 0

C. It will print NULL

D. The program's behavior is undefined and it will likely crash

Ans: D

> **Explanation:** Dereferencing a NULL pointer (as would occur here if malloc() fails) results in undefined behavior, typically a crash.

Q101 | **Basic** | *The malloc() Function*

Which of the following correctly declares and initializes a pointer to hold the result of malloc() for an integer?

A. int ptr = (int*) malloc(sizeof(int));

B. int *ptr = malloc sizeof(int);

C. int *ptr = (int*) malloc(sizeof(int));

D. *int ptr = malloc(sizeof(int));

Ans: C

> **Explanation:** The correct syntax declares ptr as an int pointer and assigns the cast result of malloc() to it.

Q102 | **Concept** | *The malloc() Function*

What kind of memory block does malloc() allocate?

A. A contiguous (single connected) block of the requested size

B. Multiple scattered blocks of varying size

C. A block that automatically resizes as needed

D. A block located only in the stack segment

Ans: A

> **Explanation:** malloc() allocates one contiguous block of memory equal to the requested size.

Q103 | **Comparison** | *The malloc() Function*

Which best distinguishes a successful malloc() call from a failed one?

A. A successful call returns NULL; a failed call returns a valid pointer

B. Both successful and failed calls return the same value

C. A successful call returns a valid non-NULL pointer; a failed call returns NULL

D. A failed call always terminates the program automatically

Ans: C

> **Explanation:** The difference is the returned pointer: valid address on success, NULL on failure.

Q104 | **Fill in the Blank** | *The malloc() Function*

To allocate memory for n integers, the size argument passed to malloc() should be _____.

A. n only

B. sizeof(int) only

C. n * sizeof(int)

D. n + sizeof(int)

Ans: C

> **Explanation:** The total memory needed for n integers is the count multiplied by the size of one integer.

Q105 | **Difficult** | *The malloc() Function*

Why is it generally unsafe to assume malloc() always succeeds in a program that allocates very large amounts of memory?

A. Because malloc() only works for small amounts of memory by design

B. Because the system may not have enough free memory, causing malloc() to return NULL

C. Because malloc() cannot allocate memory larger than 10 bytes

D. Because C does not allow large memory allocations under any circumstance

Ans: B

> **Explanation:** Large allocation requests can exceed available memory, so malloc() may fail and return NULL, making success checks essential.

Q106 | **Statement Based** | *The malloc() Function*

Which of the following is true about the memory allocated by malloc()?

A. It is automatically freed after the function that allocated it returns

B. It exists only within the block where malloc() was called

C. It remains allocated until it is explicitly freed or the program terminates

D. It is automatically converted into a static variable

Ans: C

> **Explanation:** Memory from malloc() persists beyond the function's scope until explicitly freed with free() or the program ends.

Q107 | **Scenario** | *The malloc() Function*

A programmer allocates memory with malloc() inside a function and returns the pointer to the caller. Is this valid?

A. Yes, because malloc()-allocated memory persists after the function returns, unlike local stack variables

B. No, because all memory allocated inside a function is destroyed when the function returns

C. No, because malloc() cannot be used inside functions

D. Yes, but only if the pointer is declared as static

Ans: A

> **Explanation:** Since malloc() allocates from the heap, the memory remains valid after the function returns, unlike automatic local variables.

Q108 | **Basic** | *The malloc() Function*

Which library function family does malloc() belong to?

A. Standard input/output functions

B. String handling functions

C. Standard C library dynamic memory management functions

D. Mathematical functions

Ans: C

> **Explanation:** malloc() is one of the standard C library functions for dynamic memory management, found in stdlib.h.

Q109 | Tricky | *The malloc() Function*

What is the risk of using a pointer returned by malloc() immediately after allocation without initializing its content?

A. The pointer will refer to memory with garbage (unknown) values until explicitly set

B. The compiler will refuse to compile the program

C. The pointer will automatically be set to zero

D. The pointer will point to the previous variable declared in the program

Ans: A

> **Explanation:** malloc() leaves memory uninitialized, so any values read before assignment will be indeterminate garbage.

Q110 | Difficult | *The malloc() Function*

Which of the following best explains why malloc()'s return type is void* rather than a specific type like int*?

A. Because malloc() cannot return any pointer type

B. Because void* uses less memory than other pointer types

C. Because malloc() is only used for void functions

D. So that the same function can be used to allocate memory for any data type

Ans: D

> **Explanation:** A generic void* return type allows malloc() to be used generally for any data type, with the caller casting as needed.

Chapter 4 The free() Function

30 Questions

Q111 | Purpose | *The free() Function*

What is the purpose of the free() function in C?

A. To allocate new memory dynamically

B. To print the value stored in a pointer

C. To release memory that was previously allocated dynamically, returning it to the system for reuse

D. To copy the contents of one array to another

Ans: C

> **Explanation:** free() releases previously allocated dynamic memory so it can be reused by the program or system.

Q112 | **Syntax** | *The free() Function*

What is the correct syntax for using free()?

A. free[ptr];

B. ptr = free();

C. free(ptr, size);

D. free(ptr);

Ans: D

Explanation: free() is called with the pointer to be released as its only argument, written as free(ptr);.

Q113 | **Argument** | *The free() Function*

What type of argument does free() require?

A. An integer representing the size of memory to release

B. A pointer previously returned by a dynamic memory allocation function such as malloc()

C. A string containing the variable's name

D. Two pointers to be compared

Ans: B

Explanation: free() takes a single pointer argument, which must be one returned by a memory allocation function like malloc().

Q114 | **Header** | *The free() Function*

Which header file declares the free() function?

A. stdio.h

B. stdlib.h

C. stdlib.c

D. memory.h only

Ans: B

Explanation: free(), along with malloc(), is declared in stdlib.h.

Q115 | **Concept** | *The free() Function*

What happens to the memory pointed to by a pointer after free() is called on it?

A. The memory is permanently destroyed and cannot be reused

B. The memory is automatically reallocated to the same pointer

C. The memory is released back to the system, becoming available for future allocation

D. The memory is converted into a static variable

Ans: C

Explanation: free() releases the memory block so that it can be reused for future allocations.

Q116 | **Relationship** | *The free() Function*

What is the relationship between malloc() and free()?

A. malloc() and free() perform the same operation

B. free() must always be called before malloc()

C. malloc() allocates memory, and free() is used to release memory previously allocated by malloc()

D. malloc() automatically calls free() internally after every statement

Ans: C

Explanation: free() is the counterpart to malloc(); it releases memory that malloc() (or a similar function) allocated.

Q117 | **Consequence** | *The free() Function*

What is a possible consequence of not calling free() on dynamically allocated memory that is no longer needed?

A. The program automatically stops running

B. A memory leak, where memory remains reserved even though it is no longer used

C. The unused memory is converted into a global variable

D. The compiler generates a syntax error

Ans: B

Explanation: Failing to free unused dynamically allocated memory results in a memory leak.

Q118 | **Common Mistake** | *The free() Function*

What is a common mistake when using free()?

A. Calling free() only once on a pointer

B. Calling free() more than once on the same pointer (double free)

C. Including stdlib.h before calling free()

D. Passing a pointer previously returned by malloc() to free()

Ans: B

Explanation: Calling free() twice on the same pointer, known as a double free, leads to undefined behavior.

Q119 | **Common Mistake / Dangling Pointer** | *The free() Function*

What is a “dangling pointer” in the context of free()?

A. A pointer that has never been assigned any address

B. A pointer that only stores integer values

C. A pointer that still holds the address of memory that has already been freed

D. A pointer that has been declared but not yet used

Ans: C

> **Explanation:** After free() is called, if the pointer still holds the old address and is used, it is called a dangling pointer.

Q120 | **Tricky** | *The free() Function*

What happens if a program uses (dereferences) a pointer after calling free() on it, without reassigning it?

A. The behavior is undefined and may cause unpredictable results or crashes

B. The program automatically re-allocates the memory

C. The value read will always be zero

D. The pointer becomes a null pointer automatically

Ans: A

> **Explanation:** Dereferencing a freed pointer (dangling pointer) leads to undefined behavior since the memory may have been reused.

Q121 | **Proper Usage** | *The free() Function*

What is good practice immediately after calling free(ptr)?

A. Call malloc() again with the same variable name immediately

B. Delete the variable declaration from the program

C. Call free(ptr) a second time to confirm deallocation

D. Set ptr to NULL to avoid accidentally using it as a dangling pointer

Ans: D

> **Explanation:** Setting the pointer to NULL after freeing helps prevent accidental use of a dangling pointer.

Q122 | **Statement Based** | *The free() Function*

Which of the following statements about free() is correct?

A. free() does not change the value of the pointer variable itself; it only releases the memory it points to

B. free() automatically sets the pointer variable to NULL

C. free() deletes the pointer variable from the program

D. free() always returns a new memory address

Ans: A

> **Explanation:** free() releases the memory but the pointer variable still holds the old (now invalid) address unless the programmer sets it to NULL manually.

Q123 | **Concept** | *The free() Function*

What does free(NULL) do in C?

A. It causes the program to crash

B. Nothing; it is safe and has no effect

C. It allocates a new NULL block of memory

D. It causes undefined behavior in all compilers

Ans: B

> **Explanation:** According to the C standard, calling free() with a NULL pointer is a safe no-operation.

Q124 | **Code Snippet** | *The free() Function*

What does the following code do?

```
int *p = (int*) malloc(sizeof(int));
*p = 10;
free(p);
```

A. Allocates memory but never stores or releases it

B. Releases memory before allocating it

C. Allocates memory, stores 10 in it, then releases the allocated memory

D. Causes a compilation error

Ans: C

> **Explanation:** The code allocates memory, assigns a value, and correctly frees the memory afterward.

Q125 | **Error Identification** | *The free() Function*

What is wrong with the following code?

```
int *p = (int*) malloc(sizeof(int));
free(p);
free(p);
```

A. malloc() was called with the wrong size

B. The pointer p was never assigned a value

C. free(p) is called twice on the same pointer, which is a double free error

D. free() cannot be used with integer pointers

Ans: C

> **Explanation:** Calling free() twice on the same pointer without reallocating causes undefined behavior (double free).

Q126 | **Scenario** | *The free() Function*

A program allocates memory using malloc() inside a loop but never calls free() for memory no longer needed. What is the likely result over many iterations?

A. The program's memory usage grows continuously, potentially leading to a memory leak

B. The program automatically frees old memory before each new allocation

C. The loop will not execute more than once

D. malloc() will refuse to allocate memory a second time

Ans: A

> **Explanation:** Without free(), each allocation adds to unused memory, causing a memory leak that grows with more iterations.

Q127 | **Basic** | *The free() Function*

Which of the following best describes what free() returns?

A. free() returns the address that was freed

B. free() does not return any value (it is a void function)

C. free() returns 1 on success and 0 on failure

D. free() returns the size of memory that was freed

Ans: B

> **Explanation:** free() has a void return type; it does not provide any return value.

Q128 | **Application** | *The free() Function*

In a simple C program that allocates memory for storing a value and prints it, when should free() be called?

A. Before malloc() is called

B. Immediately after declaring the pointer, before allocation

C. After the allocated memory has been used and is no longer needed, before the program ends

D. free() should never be called in a simple program

Ans: C

> **Explanation:** free() should be called once the dynamically allocated memory is no longer required, typically near the end of its use.

Q129 | **Statement Based** | *The free() Function*

Which of the following is true regarding the pointer passed to free()?

A. It should be a pointer that was returned by a memory allocation function like malloc(), and not yet freed

B. It can be any pointer, even one that was never allocated dynamically

C. It must always point to an integer

D. It must be declared as a global variable

Ans: A

> **Explanation:** Passing a pointer to free() that was not obtained from a dynamic allocation function, or freeing it twice, causes undefined behavior.

Q130 | **Fill in the Blank** | *The free() Function*

The function used to release memory previously obtained through malloc() is _____.

A. release()

B. free()

C. delete()

D. clear()

Ans: B

> **Explanation:** In C, dynamically allocated memory is released using the free() function.

Q131 | **Concept** | *The free() Function*

Why is it important to call free() on memory allocated with malloc() once it is no longer needed?

A. To increase the size of the allocated memory

B. To convert the memory into a static variable

C. To return the memory to the system so it can be reused, preventing memory leaks

D. To make the pointer point to a new random address

Ans: C

> **Explanation:** Calling free() prevents wasted, unreachable memory (memory leaks) by returning it for reuse.

Q132 | **Assertion-Reason** | *The free() Function*

Assertion (A): free(p) should be called only once for a given allocation.
Reason (R): Calling free() more than once on the same pointer results in undefined behavior.

A. A is true but R is false

B. A is false but R is true

C. Both A and R are true, and R correctly explains A

D. Both A and R are false

Ans: C

> **Explanation:** The reason (undefined behavior from double free) is exactly why free() should only be called once per allocation.

Q133 | **Program Tracing** | *The free() Function*

What is the state of the pointer p after executing free(p); if p is not manually set to NULL afterward?

A. p still holds the old address, but that memory is no longer valid to use (a dangling pointer)

B. p is automatically changed to point to new memory

C. p is automatically set to 0

D. p is deleted from the program entirely

Ans: A

> **Explanation:** free() does not change the pointer's stored address; it becomes a dangling pointer unless explicitly set to NULL.

Q134 | **Tricky** | *The free() Function*

Which of the following best explains why setting a pointer to NULL after free() is considered good practice?

A. It permanently deletes the pointer variable

B. It allocates new memory automatically

C. It prevents the pointer from being mistakenly dereferenced as a dangling pointer later in the program

D. It is required by the C compiler to avoid errors

Ans: C

> **Explanation:** Setting the pointer to NULL avoids accidental use of a now-invalid (dangling) pointer address.

Q135 | **Comparison** | *The free() Function*

How does free() differ from malloc() in terms of function signature?

A. free() takes a size and returns a pointer, while malloc() takes a pointer and returns nothing

B. Both take the same argument and return the same type

C. free() and malloc() have identical function signatures

D. free() takes a pointer and returns nothing, while malloc() takes a size and returns a pointer

Ans: D

> **Explanation:** malloc(size) returns a pointer, while free(pointer) returns void; their signatures serve opposite roles.

Q136 | **Basic** | *The free() Function*

Which of these correctly frees memory pointed to by pointer variable data?

A. free(&data);

B. data = free();

C. free(*data);

D. free(data);

Ans: D

> **Explanation:** free() takes the pointer itself (not its address or dereferenced value) as the argument.

Q137 | **Statement Based** | *The free() Function*

Which of the following is a correct pairing in dynamic memory management?

A. malloc() to release memory, free() to obtain it

B. malloc() and free() both only obtain memory

C. malloc() to obtain memory, free() to release it

D. malloc() and free() both only release memory

Ans: C

> **Explanation:** malloc() obtains memory, and free() is its counterpart used to release memory back to the system.

Q138 | **Difficult** | *The free() Function*

Which scenario represents correct and safe use of malloc() and free() together?

A. Allocate memory with malloc(), free it, then use the memory afterward

B. Call free() before calling malloc() on the same pointer

C. Call malloc() and free() on the same pointer repeatedly without using the memory in between

D. Allocate memory with malloc(), check for NULL, use the memory, then free it exactly once when done

Ans: D

> **Explanation:** The correct pattern is to allocate, verify success, use the memory appropriately, and free it exactly once when finished.

Q139 | **Scenario** | *The free() Function*

If a program frees a pointer and then reassigns it with a new call to malloc(), is this valid?

A. Yes, since the pointer variable can be reused to store a new, valid address from a new allocation

B. No, a pointer can never be reused after being freed

C. No, malloc() cannot be called again on a variable that was previously freed

D. Yes, but only if the pointer is renamed first

Ans: A

> **Explanation:** After freeing, the same pointer variable can safely be assigned a new address from another malloc() call.

Q140 | **Application** | *The free() Function*

Why is understanding free() essential when writing programs that use malloc() extensively?

A. Because free() is needed to compile any program that uses variables

B. Because free() increases the size of allocated memory

C. Because programs cannot run without calling free() at least once

D. To properly manage memory and avoid leaks or errors from mismanaged deallocation

Ans: D

> **Explanation:** Proper use of free() ensures memory allocated by malloc() is released appropriately, avoiding leaks and other memory errors.

Chapter 5 Simple Programs Using malloc() and free() 30 Questions

Q141 | **Program Tracing** | *Simple Programs Using malloc() and free()*

What is the output of the following program?

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*) malloc(sizeof(int));
    *p = 15;
    printf("%d", *p);
    free(p);
    return 0;
}
```

A. Garbage value

B. 0

C. 15

D. Compilation error

Ans: C

> **Explanation:** The program allocates memory, stores 15 in it, and prints the value before freeing the memory, so it outputs 15.

Q142 | **Error Identification** | *Simple Programs Using malloc() and free()*

What is the error in the following code?

```
int main() {
    int *p = (int*) malloc(sizeof(int));
    *p = 20;
    printf("%d", *p);
    return 0;
}
```

A. The value 20 cannot be stored in a pointer

B. malloc() is used incorrectly for integers

C. printf() cannot print pointer values

D. The allocated memory is never released using free() before the program ends

Ans: D

> **Explanation:** The code is functionally correct but fails to free the dynamically allocated memory, leading to a memory leak.

Q143 | **Correct Sequence** | *Simple Programs Using malloc() and free()*

What is the correct order of operations in a simple program using malloc() and free()?

A. Allocate memory, check for NULL, use the memory, then free the memory

B. Free the memory, allocate memory, then use it

C. Use the memory, allocate it, then check for NULL

D. Check for NULL, free the memory, then allocate it

Ans: A

> **Explanation:** The proper sequence is to first allocate, verify success, use the memory, and finally free it.

Q144 | **Program Tracing** | *Simple Programs Using malloc() and free()*

What will the following code print?

```
int *p = (int*) malloc(3 * sizeof(int));
p[0] = 1; p[1] = 2; p[2] = 3;
printf("%d %d %d", p[0], p[1], p[2]);
free(p);
```

A. 1 2 3

B. Garbage values

C. 0 0 0

D. Compilation error

Ans: A

> **Explanation:** The program allocates memory for 3 integers, assigns values 1, 2, and 3, and correctly prints them.

Q145 | **Error Identification** | *Simple Programs Using malloc() and free()*

Identify the incorrect memory-management practice in this code:

```
int *p = (int*) malloc(sizeof(int));
free(p);
*p = 5;
printf("%d", *p);
```

A. malloc() was called with an incorrect size

B. The code uses the pointer p after it has already been freed (dangling pointer usage)

C. free() was called before malloc()

D. printf() is missing format specifiers

Ans: B

> **Explanation:** Using p after free(p) accesses memory that has already been released, leading to undefined behavior.

Q146 | **Correctness / Proper Use** | *Simple Programs Using malloc() and free()*

Which of the following code snippets correctly allocates and uses memory for a single integer?

A. int *p = malloc(sizeof(int)); free(p); *p = 5; printf("%d", *p);

B. int p = (int*) malloc(sizeof(int)); *p = 5;

C. int *p = (int*) malloc(sizeof(int)); if (p != NULL) { *p = 5; printf("%d", *p); free(p); }

D. int *p; *p = 5; malloc(sizeof(int));

Ans: C

> **Explanation:** This snippet allocates memory, checks for NULL, uses the memory correctly, then frees it — the proper sequence.

Q147 | **NULL Check** | *Simple Programs Using malloc() and free()*

What should the following incomplete code check before using pointer p?

```
int *p = (int*) malloc(sizeof(int) * 100);
// missing check
p[0] = 10;
```

A. Whether p is a global variable

B. Whether p is NULL, to confirm that the allocation succeeded before use

C. Whether p was declared as static

D. Whether the printf() function is available

Ans: B

> **Explanation:** The program should check if p is NULL before using it, to handle the case where malloc() fails.

Q148 | **Program Tracing** | *Simple Programs Using malloc() and free()*

```
What is the output of the following code if malloc() succeeds?
int *p = (int*) malloc(sizeof(int));
if (p != NULL) {
    *p = 100;
    printf("%d", *p);
} else {
    printf("Allocation failed");
}
free(p);
```

A. Allocation failed

B. Garbage value

C. 100

D. 0

Ans: C

> **Explanation:** Since malloc() succeeds, the code stores 100 in the allocated memory and prints it.

Q149 | **Error Identification** | *Simple Programs Using malloc() and free()*

What mistake is present in the following code?

```
int *p = (int*) malloc(sizeof(int));
free(p);
free(p);
```

A. malloc() allocated the wrong amount of memory

B. The pointer p was declared with the wrong type

C. free() was called before malloc()

D. free(p) is called twice on the same pointer, which is a double-free error

Ans: D

Explanation: Calling free() twice on the same pointer without reallocating in between is an incorrect and undefined-behavior-causing practice.

Q150 | **Understanding Allocation** | *Simple Programs Using malloc() and free()*

In the code int *arr = (int*) malloc(5 * sizeof(int));, how many integers can be stored using arr?

A. 1

B. 5

C. 10

D. sizeof(int)

Ans: B

Explanation: The allocation reserves space for 5 integers (5 * sizeof(int) bytes), so arr can hold 5 int values.

Q151 | **Understanding Deallocation** | *Simple Programs Using malloc() and free()*

After free(arr); is executed on a previously allocated array pointer, what should NOT be done next?

A. Reassigning arr to the result of a new malloc() call

B. Setting arr to NULL

C. Declaring a new pointer variable

D. Accessing elements of arr, such as arr[0], without reallocating memory first

Ans: D

Explanation: Accessing the array after freeing it uses invalid (freed) memory, which is incorrect practice.

Q152 | **Program Tracing / Error** | *Simple Programs Using malloc() and free()*

What is the problem with the following program?

```
int main() {
    int *p;
    *p = 10;
    printf("%d", *p);
    return 0;
}
```

A. The value 10 is too large to store in an integer

B. printf() is used incorrectly

C. The program is missing only the free() function

D. The pointer p is used without first allocating memory for it using malloc()

Ans: D

> **Explanation:** p is an uninitialized pointer; dereferencing it without allocating memory leads to undefined behavior.

Q153 | **Correct Sequence Identification** | *Simple Programs Using malloc() and free()*

Which sequence of steps represents correct usage of malloc() and free() in a simple program?

A. Declare pointer, free memory, allocate memory, use memory

B. Declare pointer, allocate memory with malloc(), check for NULL, use memory, free memory

C. Use memory, declare pointer, allocate memory, free memory

D. Allocate memory, free memory, check for NULL, use memory

Ans: B

> **Explanation:** The pointer must be declared and allocated, verified, then used, and finally freed — in that specific order.

Q154 | **Program Tracing** | *Simple Programs Using malloc() and free()*

What does the following program output?

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*) malloc(sizeof(int));
    if (p == NULL) {
        printf("Memory not allocated");
    } else {
        *p = 50;
        printf("%d", *p);
    }
    free(p);
    return 0;
}
```

A. Memory not allocated

B. 50 (assuming malloc() succeeds)

C. Garbage value

D. Compilation error

Ans: B

Explanation: Since malloc() typically succeeds for such a small request, the program stores and prints 50.

Q155 | **Understanding Return Pointer** | *Simple Programs Using malloc() and free()*

In the statement int *p = (int*) malloc(sizeof(int) * 10);, what does p point to after a successful allocation?

A. The last integer in the block

B. A single integer variable only, regardless of size requested

C. Nothing until a value is assigned

D. The first byte (start) of a contiguous block of memory sized for 10 integers

Ans: D

Explanation: p receives the starting address of the allocated block, which is sized for 10 integers.

Q156 | **Error Identification** | *Simple Programs Using malloc() and free()*

What is incorrect about the following code?

```
int *p = (int*) malloc(sizeof(int));
*p = 5;
int *q = p;
free(p);
free(q);
```

A. Both p and q point to the same memory, so freeing both causes a double-free error

B. q should have been declared as a different data type

C. malloc() should have been called twice, once for each pointer

D. The value 5 cannot be shared between two pointers

Ans: A

> **Explanation:** Since q and p point to the same allocated block, calling free() on both results in freeing the same memory twice.

Q157 | **Scenario / Application** | *Simple Programs Using malloc() and free()*

A program needs to read n integers (n given by the user) and store them, then print their sum before ending. Which approach uses malloc() and free() correctly?

A. Allocate memory for n integers with malloc(), read and store the values, compute the sum, then free the memory before the program ends

B. Allocate memory for n integers only after computing the sum

C. Free the memory for n integers before reading any values

D. Avoid using malloc() and instead only use fixed-size arrays

Ans: A

> **Explanation:** The correct practice is to allocate based on n, use the memory to read and sum values, then free it once done.

Q158 | **Understanding NULL** | *Simple Programs Using malloc() and free()*

What does it mean if, after calling p = (int*) malloc(1000000000000);, the condition if (p == NULL) evaluates to true?

A. The allocation succeeded and p points to valid memory

B. The malloc() function was used with incorrect syntax

C. The variable p was never declared

D. The memory allocation failed because the system could not provide the requested amount of memory

Ans: D

> **Explanation:** A NULL result from malloc() indicates that the allocation request could not be satisfied.

Q159 | **Program Tracing** | *Simple Programs Using malloc() and free()*

```
What happens when the following code executes, assuming malloc() fails and returns
NULL?
int *p = (int*) malloc(sizeof(int) * 1000000000000);
*p = 5;
printf("%d", *p);
```

A. It prints 5 correctly

B. It prints 0

C. The program exhibits undefined behavior because it dereferences a NULL pointer

D. It automatically retries the allocation with a smaller size

Ans: C

> **Explanation:** Without a NULL check, dereferencing p (which is NULL after failed allocation) leads to undefined behavior.

Q160 | **Correct Identification** | *Simple Programs Using malloc() and free()*

Which of the following correctly frees memory allocated for an array using malloc()?

A. free(arr); where arr is the pointer originally returned by malloc()

B. free(arr[0]);

C. free(&arr);

D. Calling free(arr) before malloc(arr) in the code

Ans: A

> **Explanation:** free() should be called on the original pointer returned by malloc(), not on individual elements or its address.

Q161 | **Program Tracing** | *Simple Programs Using malloc() and free()*

```
Given the code below, what is stored in *p immediately after the malloc() call and
before any assignment?
int *p = (int*) malloc(sizeof(int));
```

A. An indeterminate (garbage) value, since malloc() does not initialize memory

B. The value 0

C. The value 1

D. The address of p itself

Ans: A

> **Explanation:** malloc() does not initialize memory, so its contents are indeterminate until a value is explicitly assigned.

Q162 | **Statement Based / Error** | *Simple Programs Using malloc() and free()*

Which of these is an example of an incorrect memory-management practice?

A. Checking for NULL right after calling malloc()

B. Calling free() exactly once after the memory is no longer needed

C. Casting the result of malloc() to the appropriate pointer type

D. Using a pointer to access memory after it has already been freed

Ans: D

> **Explanation:** Accessing memory after it has been freed is a classic memory-management error (dangling pointer usage).

Q163 | **Application Based** | *Simple Programs Using malloc() and free()*

In a program that dynamically allocates memory for an array of student marks and computes the average, where should free() be placed?

A. After the average has been computed and the array is no longer needed

B. Immediately after malloc(), before storing any marks

C. Before malloc() is called

D. free() is not needed if the program ends immediately afterward

Ans: A

> **Explanation:** free() should be called once the allocated array has served its purpose and is no longer required, i.e., after computing the average.

Q164 | **Program Tracing** | *Simple Programs Using malloc() and free()*

What is the output of the following program?

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*) malloc(2 * sizeof(int));
    p[0] = 4;
    p[1] = 6;
    printf("%d", p[0] + p[1]);
    free(p);
    return 0;
}
```

A. 10

B. 46

C. 4 6

D. Compilation error

Ans: A

> **Explanation:** p[0] and p[1] are assigned 4 and 6 respectively, and their sum, 10, is printed.

Q165 | **Error Identification** | *Simple Programs Using malloc() and free()*

What is the issue with the following program?

```
int main() {
    int *p = (int*) malloc(sizeof(int));
    free(p);
    printf("%d", *p);
    return 0;
}
```

A. malloc() was never called in this program

B. The program accesses (dereferences) the pointer p after it has already been freed

C. printf() cannot be used with pointers

D. free() was called with the wrong argument type

Ans: B

> **Explanation:** Dereferencing p after free(p) accesses invalid memory, since the memory has already been released.

Q166 | **Correct Sequence** | *Simple Programs Using malloc() and free()*

In a program that allocates memory, uses it, and then needs a new block, which sequence is correct for the pointer variable?

A. Assign the new block to the pointer before freeing the old one

B. Free the old block before assigning the pointer to a newly allocated block, to avoid losing track of the old memory

C. Never free the old block once a new one is assigned

D. Allocate the new block before allocating the old one

Ans: B

> **Explanation:** Freeing the old memory before reassigning the pointer prevents losing the reference to memory that must be released (avoiding a memory leak).

Q167 | **Program Tracing** | *Simple Programs Using malloc() and free()*

Consider the code: int *p = (int*) malloc(sizeof(int)); free(p); p = NULL;. What is the state of p after this code executes?

A. p is a null pointer and does not reference any memory, freed or otherwise

B. p still points to the freed memory block

C. p points to a newly allocated block of memory

D. p causes a compilation error

Ans: A

> **Explanation:** Setting p to NULL after freeing it clears the dangling reference, leaving p as a safe null pointer.

Q168 | **Difficult / Tracing** | *Simple Programs Using malloc() and free()*

What is the most likely outcome of running the following code, which allocates memory in a loop without ever freeing it?

```
for (int i = 0; i < 1000; i++) {
    int *p = (int*) malloc(sizeof(int) * 1000);
}
```

A. The program will continuously consume more memory, potentially leading to a memory leak or allocation failure over time

B. The program automatically frees each previous allocation before the next loop iteration

C. Memory usage will remain constant throughout the loop

D. The loop will fail to compile

Ans: A

> **Explanation:** Since each allocation inside the loop is never freed, memory usage keeps growing, which is a memory leak.

Q169 | **Scenario** | *Simple Programs Using malloc() and free()*

Which correction fixes the memory leak in a loop that repeatedly allocates memory without freeing it?

A. Move the malloc() call outside the loop but never free it

B. Call free() on the pointer at the end of each loop iteration, after it is no longer needed

C. Remove the malloc() call and replace it with printf()

D. Call malloc() twice in each iteration instead of once

Ans: B

> **Explanation:** Adding a matching free() call within each iteration ensures memory allocated inside the loop is properly released.

Q170 | **Program Tracing** | *Simple Programs Using malloc() and free()*

What is the output of the following program?

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int*) malloc(sizeof(int));
    if (p == NULL) {
        printf("Failed");
        return 1;
    }
    *p = 7;
    printf("Value: %d", *p);
    free(p);
    return 0;
}
```

A. Failed

B. Value: 0

C. Compilation error

D. Value: 7

Ans: D

Explanation: Since malloc() succeeds for this small allocation, the program stores 7 and prints "Value: 7".